In [2]:
```
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report
)

# 2. Load the Pima Indians Diabetes dataset
url = "https://raw.githubusercontent.com/plotly/datasets/master/diabetes.

df = pd.read_csv(url)

print("Dataset Shape:", df.shape)
print("\nFirst Five Rows:")
print(df.head())

# 3. Separate input features and target
X = df.drop("Outcome", axis=1).copy()
y = df["Outcome"]

print("\nTarget Distribution:")
print(y.value_counts())

# 4. Replace invalid zero measurements with missing values
columns_to_fix = [
    "Glucose",
    "BloodPressure",
    "SkinThickness",
    "Insulin",
    "BMI"
]

X[columns_to_fix] = X[columns_to_fix].replace(0, np.nan)

# 5. Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

print("\nTraining Samples:", X_train.shape[0])
print("Testing Samples:", X_test.shape[0])

# 6. Logistic Regression WITHOUT feature scaling
model_without_scaling = Pipeline([
```

```
    ("imputer", SimpleImputer(strategy="median")),
    ("classifier", LogisticRegression(max_iter=5000, random_state=42))
])

model_without_scaling.fit(X_train, y_train)

pred_without_scaling = model_without_scaling.predict(X_test)

# 7. Logistic Regression WITH feature scaling
model_with_scaling = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler()),
    ("classifier", LogisticRegression(max_iter=5000, random_state=42))
])

model_with_scaling.fit(X_train, y_train)

pred_with_scaling = model_with_scaling.predict(X_test)

# 8. Define a function to evaluate model performance
def evaluate_model(name, y_true, y_pred):
    return {
        "Model": name,
        "Accuracy": accuracy_score(y_true, y_pred),
        "Precision": precision_score(
            y_true, y_pred, zero_division=0
        ),
        "Recall": recall_score(
            y_true, y_pred, zero_division=0
        ),
        "F1-Score": f1_score(
            y_true, y_pred, zero_division=0
        )
    }

# 9. Compare performance metrics
results = []

results.append(
    evaluate_model(
        "Without Scaling",
        y_test,
        pred_without_scaling
    )
)

results.append(
    evaluate_model(
        "With Scaling",
        y_test,
        pred_with_scaling
    )
)

results_df = pd.DataFrame(results)

print("\n========== PERFORMANCE COMPARISON ==========")
print(results_df.round(4).to_string(index=False))

# 10. Display confusion matrix without scaling
```

```
print("\nConfusion Matrix - Without Scaling:")
print(confusion_matrix(y_test, pred_without_scaling))

# 11. Display confusion matrix with scaling
print("\nConfusion Matrix - With Scaling:")
print(confusion_matrix(y_test, pred_with_scaling))

# 12. Classification report without scaling
print("\nClassification Report - Without Scaling:")
print(classification_report(
    y_test,
    pred_without_scaling,
    target_names=["No Diabetes", "Diabetes"],
    zero_division=0
))

# 13. Classification report with scaling
print("\nClassification Report - With Scaling:")
print(classification_report(
    y_test,
    pred_with_scaling,
    target_names=["No Diabetes", "Diabetes"],
    zero_division=0
))

# 14. Identify the model with the higher F1-score
best_index = results_df["F1-Score"].idxmax()
best_model_name = results_df.loc[best_index, "Model"]

print("\nModel with the Higher F1-Score:", best_model_name)

print("\nExperiment Completed Successfully.")
```

```
Dataset Shape: (768, 9)

First Five Rows:
   Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin   BMI  \
0            6      148             72             35        0  33.6
1            1       85             66             29        0  26.6
2            8      183             64              0        0  23.3
3            1       89             66             23       94  28.1
4            0      137             40             35      168  43.1

   DiabetesPedigreeFunction  Age  Outcome
0                     0.627   50        1
1                     0.351   31        0
2                     0.672   32        1
3                     0.167   21        0
4                     2.288   33        1

Target Distribution:
Outcome
0    500
1    268
Name: count, dtype: int64

Training Samples: 614
Testing Samples: 154

========== PERFORMANCE COMPARISON ==========
          Model  Accuracy  Precision  Recall  F1-Score
Without Scaling    0.7013      0.587     0.5    0.5400
   With Scaling    0.7078      0.600     0.5    0.5455

Confusion Matrix - Without Scaling:
[[81 19]
 [27 27]]

Confusion Matrix - With Scaling:
[[82 18]
 [27 27]]

Classification Report - Without Scaling:
              precision    recall  f1-score   support

 No Diabetes       0.75      0.81      0.78       100
    Diabetes       0.59      0.50      0.54        54

    accuracy                           0.70       154
   macro avg       0.67      0.66      0.66       154
weighted avg       0.69      0.70      0.70       154


Classification Report - With Scaling:
              precision    recall  f1-score   support

 No Diabetes       0.75      0.82      0.78       100
    Diabetes       0.60      0.50      0.55        54

    accuracy                           0.71       154
   macro avg       0.68      0.66      0.67       154
weighted avg       0.70      0.71      0.70       154
```

```
Model with the Higher F1-Score: With Scaling

Experiment Completed Successfully.
```

In [ ]: